Chapter X: Collections of User Defined Types

Discussion of all data structures in earlier chapters focused on storing some primitive type data in the collection type data structures. It is obvious that one can store instances of user defined types in all of arrays, lists, stack, queues, hash-tables, trees, and graphs. We will then have an array of some custom type, or a list/tree/graph node holding custom data, and so forth. The deeper question here is how the data is laid out in memory when something other than some primitive value is the element type of a collection. In this chapter, we first discuss the layout of a user defined type in memory then the layout and access of collection of that type. The chapter ends with a discussion on memory leakage and its cleanup strategies. The discussion of this chapter uses arrays as the reference collection type. However, understand that the discussion is valid for all other collection types.

Memory Allocation of a Class Instance

Assume for a reference we have a user defined type (commonly known as a Class) for representing students as follows:

```
class Student {
        int id;
        float cgpa;
        int batch;
        Boolean local;
}
```

Assuming integers and floats requiring 4, 8 bytes and a Boolean requiring 1 bit of space in memory respectively; the total memory requirement for a Student class instance (a.k.a object) is 4 + 8 + 4 + 1 = 17 bytes. It may appear surprising that the memory requirement is not $16\frac{1}{8}$. The reason is that the wiring of the RAM circuit for data read-write lines only allow reading/writing data from byte addresses of RAM. Thus, there is no point of allocating fraction of a byte for a property as other objects cannot use the remaining of the fraction[1]. In fact, the compiler generated executable code requests 17 consecutive bytes from the RAM each time a student object is created. Furthermore, the properties of the class instances are stored exactly in the same order they have been declared in the class definition. That is, suppose a Student variable is 'selim' then 'x = selim.yearOfAdmission' will literally read 4 bytes of data from 12 bytes distance of the starting point of selim and copy it to the location referred by variable 'x'.

Now one can have properties of other user defined types underneath a class definition. Consider the following configuration. Here an uneven cubic volume is defined in terms of six 2D surfaces and each surface is defined by its four endpoints.

class Volume {
Surface[6] surfaces;
Boolean isConvex;
}
	class Surface {
	Point[4] endPoints;
	}
		class Point {
		int x;
		int y;
		}

[1] There is an exception to this rule for space-optimized packed data types such as a C union.

For a definition like the above, a typical compiler actually flattens the definition of the Volume class into one having 6 * 4 * 2 = 48 integers + 1 Boolean. Then the overall memory requirement 48 * 4 + 1 = 193 bytes. Suppose one creates a variable of Volume type named 'myStrangeBox' then consecutive 193 bytes will be allocated for that variable. Not so strangely, accessing properties of the variables follow the same protocol of array element access. For example, if we have 'p = myStrangeBox.surfaces[3].endPoints[1].y' as a statement in the program then it means, the executable will read 4 bytes from index 3 * (2 * 2) + 1 * 2 + 1 = 15 from starting point of 'myStrangeBox', which is equivalent to 15 * 4 = 60th byte address away from the starting byte of the variable. The bottom line is that the only thing that actually gets allocated in the memory are primitive data types and a complex object is actually an array of primitive types in the hardware. We again learn the primacy of arrays in computer science. Aren't arrays marvelous!

Collections of Class Instances

From the memory layout examples of class instances of the above, one can understand what should be the memory layout for a collection of instances of a class such as Volume or Student. For example, if one creates a dynamic array of students as follows 'Student[] students = new Students[n]' where n is an integer variable defined earlier in the program, then consecutive n * 17 bytes will be allocated for the students array where each subsequent instance of a student will be 17 bytes apart in the RAM from the previous instance.

Dealing with Variable Length Data Types

Things become more complicated when we allow properties of some variable length data types inside a user defined custom type. For example, a Student type can have a student name and an array/list of course registrations inside it. Then the logic of fixed length allocation for a student in the memory falls apart as different students' names can be of different length and their number of courses can also vary. This apparent difficulty is solved using an ingenious trick, called pointers. We will understand pointers by investigating String data type. For that assume the redefined Student class is as follows:

```
class Student {
        int id;
        String name;
        float cgpa;
        int batch;
        Boolean local;
        String address;
}
```

Strings

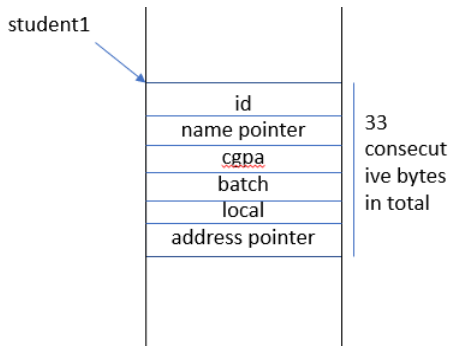
Strings are basically an array of characters (according to the Unicode standard, a character is 2 bytes long). Suppose we again create a Student class instance named 'student1' and set

```
student1.name = "Selim Khan"
student1.address = "66 Mohakhali, Dhaka"
```

Then we need 11 characters for the name attribute and 19 characters for the address attribute. Thus, the executable must request consecutive 11 and 19 characters from RAM for storing these two attributes when these two statements are executed. Actually, the space requirement for these two attributes is a bit higher. In languages like C and FORTRAN, the executable asks for one more character in each string to store a special character '\0' at the end. This character serves as the end marker for the strings. In

languages like Java and Python, an integer at the beginning of the string stores its length in terms of the number of characters. Anyway, the key concern here is that the actual space requirement for student1 is not known before the two statements assigning the name and address to student1 are executed. Then how do we create a variable like student1 in the first place? How can we even create an array of Students if we need one?

This is where the marvelous idea of a pointer variable comes in. A pointer is a data type that holds the address of a memory location in it. In older 32-bit machines, a pointer was 32 bits = 4 bytes = 1 integer. In the present day 64-bit machine, a pointer is 64 bits = 8 bytes = 1 long integer. The reason is that a memory address can be from 0 to $2^{64} - 1$ in a 64-bit machine and from 0 to $2^{32} - 1$ in a 32-bit machine. Now, when the student1 variable is created, the memory is allocated as follows.



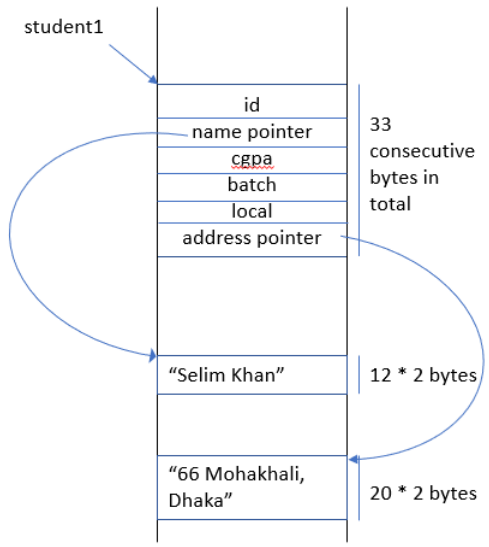
Therefore, the original student1 object occupies 33 consecutive bytes in the RAM. The pointers for the name and address attributes hold invalid values to indicate that these values have not been initialized.

The Curious Case of None/Null Pointer

Most of us are familiar with invalid object references. In different languages, the special invalid reference is recognized by different names such as, nill, null, None. Actually, an invalid object reference is nothing but a pointer variable/property where the address stored in the pointer is invalid. However, given a pointer is an integer long/normal, it must hold an integer value and will result in referring to a valid RAM location no matter what. Then how can we have an invalid value?

The wonderful idea of invalid pointers is that, languages reserve a special memory address where it will never try to store any object. This reserved address is typically 0th byte of the RAM that is address zero. That is, if the value of a pointer is zero, then the pointer is invalid.

When any statement assigning some value to any of the string properties executes, then the program gets enough consecutive space needed for holding the corresponding string and update the pointer attribute to hold the address of the beginning position of the string. So, after the previous two statements assigning values for the name and address attributes, the structure of the student should look something like as follows.



If the program changes the value of either of these properties then new free consecutive memory cells are found to hold the new values and the pointers' referred address is updated.

Collections as nested properties

From the discussion of string properties inside a user defined type, it is obvious how properties of a collection type are stored within an object instance. For example, if our previous Student class has an array of GPAs from earlier semesters as a property then each root Student instance will hold one more pointer where this new pointer will hold the address of the beginning of the array of floats where the GPAs have been stored. Similarly, for other collection types such as list, tree, or graph; the root object only holds the pointers to the beginning of the respective collection types. To put it together, pointers are the reason we can have dynamic memory allocations and support variable length data types in programming languages.

Arrays of Class Instances

The general principle of memory allocation for an array of user defined type is that all nested collection of the user defined type and all other variable length properties are replaced with pointers. Then the user defined type is flattened to find out the total memory requirement including other nested user defined properties (as in the case of the Volume class explained earlier). Then the array is allocated a contiguous memory bytes of size = number of items in the array * sizeOf(the flattened user defined type). Memory allocations for variable length properties and nested collections of the array items are no way linked with the array allocation in the memory. Rather, they are allocated and reallocated as their values being assigned and updated and the pointers in the array items get updated every time the allocation is changed for such a property.

It should be apparent from the above discussion that if we allocate an array of some collection types, for example, an array of hash-tables or an array of lists, we actually allocate an array of pointers.

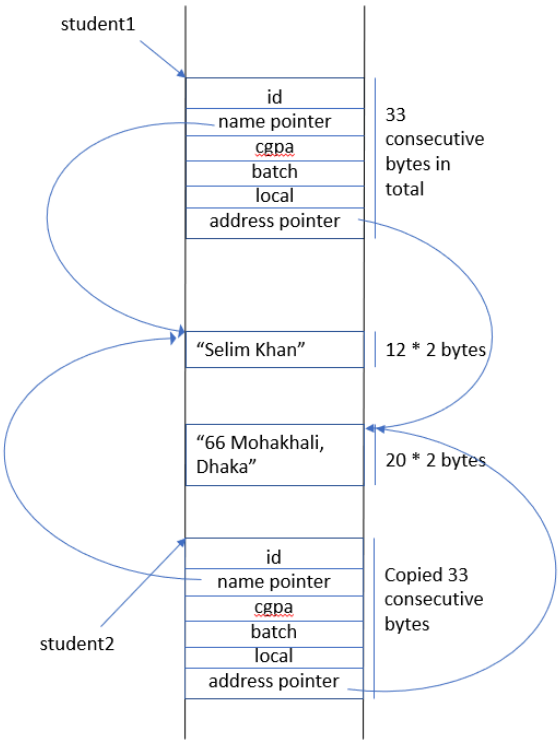
Shallow Copy and Deep Copy of Objects

We understand that when we design user defined data types, there can be a hierarchy of primitive properties, collections, and other user defined types inside a class instance variable. So, it is important to

understand what happens when we assign a class instance to another instance. Going back to our previous Student class definition and the student1 class instance, suppose we have the following statement in our program:

```
Student student2 = student1
```

Then the following will happen in the memory:



Literally the 33 bytes of student1 will be copied in student2. Notice that the strings are not replicated but their pointers are copied. So, the two Student instances refer to the same memory locations and consequently have the same strings as their name and address attributes.

If the Student class had a GPAs array as we discussed earlier then the array will be the same for both student1 and student2 as their GPAs pointers will refer to the same memory location. This is why, when an object is copied in another object, you can modify the content of nested collections through any of the two objects.

However, if you choose a new allocation for a nested collection in one object, or assign a new value to a string-type property in one then the collections and strings diverge as the pointers are no longer the same in both objects.

This whole mechanism of copying object instances is called shallow coping as the copying process does not go deep enough to separate a copy from its source. If programmers need a complete independent instance as a copy, then they need to create a new object and painstakingly redo all memory allocations and copy all values from the source to the copy. This hard process is called deep copying.

Object Destruction and Memory Reclamation

In a large program, it may be tricky to track all shared references and all nested objects. This is the main reason, programming languages that focus on application programming with ease, such as, Java, C#, and Python; do not allow programmers to define pointer variables and to manage memory programmatically. Instead, they have a runtime module typically called the Garbage Collector, that keeps track of pointer references and reclaims memory for an object/property when there is no reference to it. However, to operate a module like a garbage collector, the runtime engine periodically halts the execution of user code, and scans memory allocations to reclaim all unreferenced objects (called garbage). That hurts performance. This is the only way to do this, as if the garbage collector is run every time a dynamic property or collection is changed then the slowness of the user program would be unbearable.

In fact, due to the slowness of garbage collection, many languages, for example, Java, do not run garbage collector at all until the memory occupancy of the program becomes too high, so much so, that without collecting the garbage, there is a risk of crushing the program due to memory runout. Now this approach has a pitfall also. If the runtime is not frequently collecting the garbage, then every new memory allocation would take more time to find free slots for new data.

To simplify garbage collection task, some languages without pointers offer programmers to write a destructor function for their custom types. The purpose of a destructor function is to let the programmer specify what other nested things of the class instance should be cleaned from the memory when the class instance is garbage collected.

Note that when someone writes mission or performance critical software such as operating systems, device drivers, embedded systems code, real-time controllers then relinquishing control of the memory to the language runtime and suffering the risk of garbage collector abruptly halting your program is unbearable. That is why these things are written using languages that make the programmers manage everything, such as, C, FORTRAN, and Assembly.

The Curious Case of Slow Java Arrays and Strings

Java and several other application programming languages store the length of a string at the beginning of the string in memory. Similarly, it also stores the dimensionality and dimension lengths of an array at the beginning of the array. Every 'ch = str.chartAt(i)' or 'x = array[i]' type expression and statements in Java is actually wrapped inside a if-else block as follows:

```
If (i >= 0 && I <= array.length) {
        // the byte code for reading the array/string element
} else {
        // byte code for throwing a runtime exception
}
```

Thus, actually an array/string access in Java has two Boolean expression evaluations and doing a conditional jump beside the original access.

The philosophy of languages like C and FORTRAN is that programmers allocated the arrays/strings so they are sensible enough to have an index that is within the proper range. So, there is no condition checking of the above types in these languages. This is why C and FORTRAN are almost 3 times faster than Java (similarly of Python) for basic array/string operations.